

Total Marks: 03

Obtained Marks: _____

AI LAB

Assignment # 01

Last Date of Submission: 29 Oct 2026

Submitted To: Ms. Zahra Khalid Butt

Student Name: Akmal Ali

Registration Number: 25108108

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Instructions: Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.

CLO – 1

Lab Tasks: Python Function Arguments

Lab Task 1: Keyword Arguments Write a Python program to store and display a student's name, age, and department using keyword arguments. Pass the arguments in a different order when calling the function.

Solution:

```
[3]: def studentData(name, age, department):  
      print("Student Name:", name)  
      print("Age:", age)  
      print("Department:", department)  
  
      studentData(department="Artificial Intelligence", name="Akmal Ali", age=20)  
  
      Student Name: Akmal Ali  
      Age: 20  
      Department: Artificial Intelligence
```

Lab Task 2: Default Arguments Write a Python program to calculate the final bill amount by applying a tax rate of 5% by default. Allow the user to provide a different tax rate when required. Display the final bill amount in both cases.

Solution:

```
[5]: def cal_bill(amount, tax_rate=5):  
      tax = amount * tax_rate / 100  
      final_amount = amount + tax  
      return final_amount  
  
      bill = 1000  
      print("Bill with default tax 5%:", cal_bill(bill))  
      print("Bill with custom tax 10%:", cal_bill(bill, 10))  
  
      Bill with default tax 5%: 1050.0  
      Bill with custom tax 10%: 1100.0
```

Lab Task 3: Arbitrary Arguments (*args) Write a Python program to calculate the total shopping cost for multiple items. Allow the user to pass any number of item prices and display the total cost.

Solution:

```
[6]: def shoppingCost(*prices):  
    total = sum(prices)  
    return total  
  
print("Total Cosat:", shoppingCost(500, 250, 1000, 300))  
  
Total Cosat: 2050
```

Lab Tasks – Tuples

Task 1: Student Course Record

A university stores a student's basic course information in a tuple:

student = ("Ali", "BSCS", 3, "Database Systems")

Write a Python program to:

1. Display the student's name and program using indexing.
2. Display the last two elements using slicing.
3. Check whether "Database Systems" is present in the tuple.

Solution:

```
•[7]: student = ("Ali", "BSCS", 3, "Database Systems")  
  
print("Student Name:", student[0])  
print("Program:", student[1])  
  
print("Last Two Elements:", student[2:])  
print("Database Systems" in student)  
  
Student Name: Ali  
Program: BSCS  
Last Two Elements: (3, 'Database Systems')  
True
```

Task 2: Online Store Products

An online store stores the prices of five products in a tuple:

prices = (1200, 850, 1500, 950, 2000)

Write a Python program to:

1. Display the highest and lowest price.
2. Calculate the total price of all products.
3. Sort the prices in ascending order.

Solution:

```
[8]: prices = (1200, 850, 1500, 950, 2000)

print("Highest Price:", max(prices))
print("Lowest Price:", min(prices))

print("Total Price:", sum(prices))

sortedPrice = sorted(prices)
print("Prices in Ascending Order:", sortedPrice)

Highest Price: 2000
Lowest Price: 850
Total Price: 6500
Prices in Ascending Order: [850, 950, 1200, 1500, 2000]
```

Task 3: Hospital Appointment Days

A hospital stores the available appointment days in a tuple:

days = ("Monday", "Wednesday", "Friday", "Wednesday", "Saturday") Write a

Python program to:

1. Check whether "Friday" is available.
2. Find how many times "Wednesday" appears.
3. Find the position of the first occurrence of "Saturday".
4. Display the appointment days using a loop.

Solution:

```
[9]: days = ("Monday", "Wednesday", "Friday", "Wednesday", "Saturday")
print("Friday available:", "Friday" in days)
print("Wednesday appears:", days.count("Wednesday"), "times")
print("Position of Saturday:", days.index("Saturday"))
print("Appointment Days:")

for day in days:
    print(day)

Friday available: True
Wednesday appears: 2 times
Position of Saturday: 4
Appointment Days:
Monday
Wednesday
Friday
Wednesday
Saturday
```

Task 4: Bus Route Management

A transport service stores the stops of a bus route in a tuple:

stops = ("Islamabad", "Rawalpindi", "Taxila", "Wah", "Hasan Abdal") Write a

Python program to:

1. Display the first and last stops using indexing.
2. Display the route in reverse order using slicing.
3. Add a new stop "Attock" by creating a new tuple through concatenation.
4. Display all stops using a loop.

Solution:

```
[10]: stops = ("Islamabad", "Rawalpindi", "Taxila", "Gilgit", "Hasan Abdal")
      print("First Stop:", stops[0])
      print("Last Stop:", stops[-1])

      print("Reverse Route:", stops[::-1])

      stops = stops + ("Attock",)
      print("Updated Route:", stops)

      print("All Stops:")
      for stop in stops:
          print(stop)

First Stop: Islamabad
Last Stop: Hasan Abdal
Reverse Route: ('Hasan Abdal', 'Wah', 'Taxila', 'Rawalpindi', 'Islamabad')
Updated Route: ('Islamabad', 'Rawalpindi', 'Taxila', 'Wah', 'Hasan Abdal', 'Attock')
All Stops:
Islamabad
Rawalpindi
Taxila
Wah
Hasan Abdal
Attock
```

Lab Tasks – Sets

Task 1: University Course Registration

A university maintains two sets of students enrolled in **Python** and **Database Systems**:

python_students = {"Ali", "Sara", "Ahmed", "Hina", "Usman"}

db_students = {"Ahmed", "Hina", "Bilal", "Usman", "Fatima"}

Write a Python program to:

1. Add "Ayesha" to the Python students set.
2. Remove "Sara" from the Python students set
3. Attempt to remove "Zain" from the Database Systems students set without generating an error.
4. Display the updated sets.

Solution:

```
[12]: python_students = {"Ali", "Sara", "Ahmed", "Hina", "Usman"}
      db_students = {"Ahmed", "Hina", "Bilal", "Usman", "Fatima"}

      python_students.add("Ayesha")
      python_students.remove("Sara")
      db_students.discard("Zain")
      print("Python Students:", python_students)
      print("Database Systems Students:", db_students)

Python Students: {'Ayesha', 'Usman', 'Ahmed', 'Ali', 'Hina'}
Database Systems Students: {'Usman', 'Fatima', 'Ahmed', 'Bilal', 'Hina'}
```

Task 2: Student Attendance Analysis

A teacher records students who attended classes on **Monday** and **Wednesday**:

monday = {"Ali", "Sara", "Ahmed", "Hina", "Usman"}

wednesday = {"Sara", "Ahmed", "Hina", "Bilal", "Fatima"} Write a

Python program to determine:

- a) Students who attended both days.
- b) Students who attended at least one of the two days.
- c) Students who attended only on Monday.
- d) Students who attended only on Wednesday.
- e) Students who attended exactly one of the two days.

Solution:

```
5]: monday = {"Ali", "Sara", "Ahmed", "Hina", "Usman"}
   wednesday = {"Sara", "Ahmed", "Hina", "Bilal", "Fatima"}

   both_days = monday.intersection(wednesday)
   print("Attended Both Days:", both_days)

   at_least_one = monday.union(wednesday)
   print("Attended At Least One Day:", at_least_one)

   only_monday = monday.difference(wednesday)
   print("Only Monday:", only_monday)

   only_wednesday = wednesday.difference(monday)
   print("Only Wednesday:", only_wednesday)

   exactly_one = monday.symmetric_difference(wednesday)
   print("Exactly One Day:", exactly_one)

Attended Both Days: {'Ahmed', 'Sara', 'Hina'}
Attended At Least One Day: {'Usman', 'Fatima', 'Ahmed', 'Bilal', 'Hina', 'Sara', 'Ali'}
Only Monday: {'Ali', 'Usman'}
Only Wednesday: {'Bilal', 'Fatima'}
Exactly One Day: {'Usman', 'Fatima', 'Ali', 'Bilal'}
```

Task 3: Library Book Management

A library receives the following list of borrowed books:

```
books = ["Python", "DBMS", "OOP", "Python", "DLD", "DBMS", "AI"]
```

Write a Python program to:

- Convert the list into a set.
- Display the books after removing duplicate entries.
- Add "Machine Learning" to the collection.
- Remove "DLD" from the collection.
- Attempt to remove "Java" without generating an error.
- Display the final collection.

Solution:

```
[17]: books = ["Python", "DBMS", "OOP", "Python", "DLD", "DBMS", "AI"]

book_set = set(books)

print("Books after removing duplicates:", book_set)

book_set.add("Machine Learning")

book_set.remove("DLD")

book_set.discard("Java")

print("Final Collection:", book_set)

Books after removing duplicates: {'DLD', 'DBMS', 'AI', 'Python', 'OOP'}
Final Collection: {'DBMS', 'AI', 'Python', 'Machine Learning', 'OOP'}
```

Task 4: Online Store Product Analysis

Two customers purchased the following products:

```
customer_A = {"Laptop", "Mouse", "Keyboard", "USB", "Headphones"}
```

```
customer_B = {"Laptop", "Webcam", "Keyboard", "Printer", "USB"}
```

Write a Python program to:

1. Display the products purchased by both customers.
2. Display all unique products purchased by the two customers.
3. Display the products purchased only by Customer A.
4. Display the products purchased only by Customer B.
5. Display the products purchased by only one customer.
6. Display all products purchased by Customer A one by one.

Solution:

```
customer_A = {"Laptop", "Mouse", "Keyboard", "USB", "Headphones"}
customer_B = {"Laptop", "Webcam", "Keyboard", "Printer", "USB"}

both = customer_A.intersection(customer_B)
print("Purchased by Both:", both)

all_products = customer_A.union(customer_B)
print("All Unique Products:", all_products)

only_A = customer_A.difference(customer_B)
print("Only Customer A:", only_A)

only_B = customer_B.difference(customer_A)
print("Only Customer B:", only_B)

only_one = customer_A.symmetric_difference(customer_B)
print("Purchased by Only One Customer:", only_one)

print("Customer A Products:")

for product in customer_A:
    print(product)
```

```
Purchased by Both: {'Laptop', 'USB', 'Keyboard'}
All Unique Products: {'Mouse', 'Keyboard', 'Headphones', 'Laptop', 'USB', 'Printer', 'Webcam'}
Only Customer A: {'Mouse', 'Headphones'}
Only Customer B: {'Printer', 'Webcam'}
Purchased by Only One Customer: {'Printer', 'Mouse', 'Webcam', 'Headphones'}
Customer A Products:
Laptop
USB
Mouse
Keyboard
Headphones
```

Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University